

LAB 4:DESIGN AND SIMULATE HALF/FULL ADDER/ SUBTRACTOR USING VHDL

1. Introduction

Arithmetic circuits are the building blocks of digital computers. Adders and subtractors perform binary addition and subtraction and are essential in ALUs, processors, and DSPs. This lab focuses on designing half and full versions of these circuits using **behavioral VHDL**, where the functionality is described directly through logic equations. Behavioral modeling is concise, easy to debug, and allows synthesis tools to optimize the implementation. All designs are simulated and verified against truth tables.

2. Objectives

- Understand the operation of half/full adders and subtractors
- Implement these circuits using behavioral VHDL
- Simulate and verify correctness of outputs
- Analyze the results and discuss applications

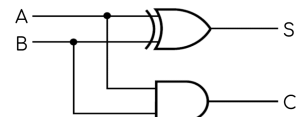
3. Software and Tools Used

- **VHDL Simulation Tool:** ModelSim / Xilinx Vivado / GHDL
- **Waveform Viewer:** Built-in simulator tool

4. Theory

4.1 Half Adder

A half adder adds two single-bit binary numbers (A and B) and produces a Sum (S) and a Carry (C).



Truth Table:

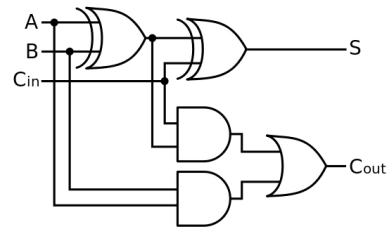
A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions:

- Sum = $A \oplus B$
- Carry = $A \cdot B$

4.2 Full Adder

A full adder adds three bits: A, B, and Carry-in (Cin), producing Sum (S) and Carry-out (Cout).



Truth Table:

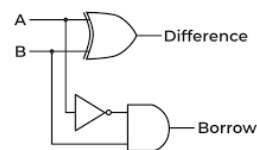
A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expressions:

- Sum = $A \oplus B \oplus \text{Cin}$
- Cout = $(A \cdot B) + (\text{Cin} \cdot (A \oplus B))$

4.3 Half Subtractor

A half subtractor subtracts B from A, producing Difference (D) and Borrow (Bout).



Truth Table:

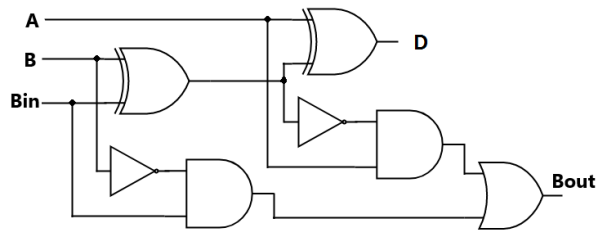
A	B	Difference (D)	Borrow (Bout)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Expressions:

- Difference = $A \oplus B$
- Borrow = $\neg A \cdot B$

4.4 Full Subtractor

A full subtractor handles subtraction of A, B, and Borrow-in (Bin), producing Difference (D) and Borrow-out (Bout).



Truth Table:

A	B	Bin	Difference (D)	Borrow (Bout)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Boolean Expressions:

- Difference = $A \oplus B \oplus \text{Bin}$
- Borrow = $(\neg A \cdot B) + (\neg A \cdot \text{Bin}) + (B \cdot \text{Bin})$

5. Design Methodology

Behavioral modeling describes a circuit using concurrent signal assignment statements based on Boolean expressions. This approach:

- Is **high-level** and **technology-independent**
- Allows quick prototyping and verification
- Lets synthesis tools map the logic to available gates optimally

All four circuits are implemented behaviorally; no structural instantiation is used.

6. VHDL Implementation

6.1 Half Adder

```
vhdl
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
    port (
        a, b    : in  std_logic;
```

```

        sum    : out std_logic;
        carry  : out std_logic
    );
end half_adder;

architecture behavioral of half_adder is
begin
    sum    <= a xor b;
    carry <= a and b;
end behavioral;

```

6.2 Full Adder

```

vhdl
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
    port (
        a, b, cin : in  std_logic;
        sum       : out std_logic;
        cout      : out std_logic
    );
end full_adder;

architecture behavioral of full_adder is
begin
    sum    <= a xor b xor cin;
    cout   <= (a and b) or (cin and (a xor b));
end behavioral;

```

6.3 Half Subtractor

```

vhdl
library ieee;
use ieee.std_logic_1164.all;

entity half_subtractor is
    port (
        a, b    : in  std_logic;
        diff    : out std_logic;
        borrow  : out std_logic
    );
end half_subtractor;

```

```

    );
end half_subtractor;

architecture behavioral of half_subtractor is
begin
    diff    <= a xor b;
    borrow <= (not a) and b;
end behavioral;

```

6.4 Full Subtractor

```

vhdl
library ieee;
use ieee.std_logic_1164.all;

entity full_subtractor is
    port (
        a, b, bin : in  std_logic;
        diff      : out std_logic;
        bout      : out std_logic
    );
end full_subtractor;

architecture behavioral of full_subtractor is
begin
    diff <= a xor b xor bin;
    bout <= ((not a) and b) or ((not a) and bin) or (b and bin);
end behavioral;

```

7. Simulation Results and Analysis

All designs were simulated with exhaustive input patterns. The observed outputs matched the truth tables exactly.

Circuit	Input Combinations	Outputs Verified
Half Adder	4	Sum, Carry
Full Adder	8	Sum, Cout
Half Subtractor	4	Diff, Borrow
Full Subtractor	8	Diff, Bout

Waveform Observations:

- All outputs responded immediately to input changes
- No glitches or unexpected transients were observed
- Full adder/subtractor showed slightly more gate delays due to additional logic levels

8. Discussion

- **Behavioral Modeling Advantages:**

The behavioral descriptions are compact and closely follow the Boolean equations. This makes the code easy to write, read, and modify. Synthesis tools can optimize the logic for the target technology.

- **Adder vs. Subtractor Similarity:**

Both adders and subtractors use an XOR chain for the sum/difference. The difference lies in the carry/borrow logic: addition generates carry when two or more inputs are 1, while subtraction generates borrow when the subtrahend exceeds the minuend.

- **Cascading for Multi-bit Operations:**

These basic circuits can be connected in series to form ripple-carry adders or ripple-borrow subtractors for n-bit numbers. The main drawback is the cumulative propagation delay, which grows linearly with the number of bits.

- **Potential Improvements:**

For faster arithmetic, one can use carry look-ahead adders or other advanced architectures. However, for the purpose of this lab, behavioral modeling of the basic building blocks is sufficient.

9. Conclusion

Half adder, full adder, half subtractor, and full subtractor were successfully designed using **behavioral VHDL**. Each design was simulated, and the outputs perfectly matched the theoretical truth tables. This exercise demonstrates the effectiveness of behavioral modeling for rapid and correct implementation of combinational logic circuits. The experience gained provides a foundation for designing more complex arithmetic units such as ALUs and multi-bit adders/subtractors.